CENG 111 Algoritmalar ve Programlama 6: Fonksiyonlar

Öğr.Gör. Şevket Umut ÇAKIR

Pamukkale Üniversitesi

Hafta 6

Anahat

1. Giriş
2. C'de Programları Modülerleştirme
3. Math Kütüphane Fonksiyonları
4. Fonksiyonlar
5. Fonksiyon Tanımları
6. Fonksiyon Prototipleri: Derin Bir Bakış
7. Fonksiyon Çağrı Yığıtı ve Yığıt Çerçeveleri
8. Başlıklar
9. Argümanları Değer ve Referans Olarak Geçme
10. Rastgele Sayı Üretimi
11. Rastgele Sayı Vaka Çalışması: Bir Casino Oyunu

Giriş

- Gerçek dünya problemleri, bu ünitelerde gördüğümüzden daha karmaşık yapılara sahiptir
- Programları küçük parçalar halinde oluşturmak geliştirme ve bakım açısından daha iyi olmaktadır
  - Tüm programı yönetmektense parçaları yönetmek daha kolay
- Bu yöntem böl ve yönet(divide and conquer) adını alır
- Bu ünitede programı küçük parçalara ayırmak için kullanacağımız fonksiyonlar işlenecektir

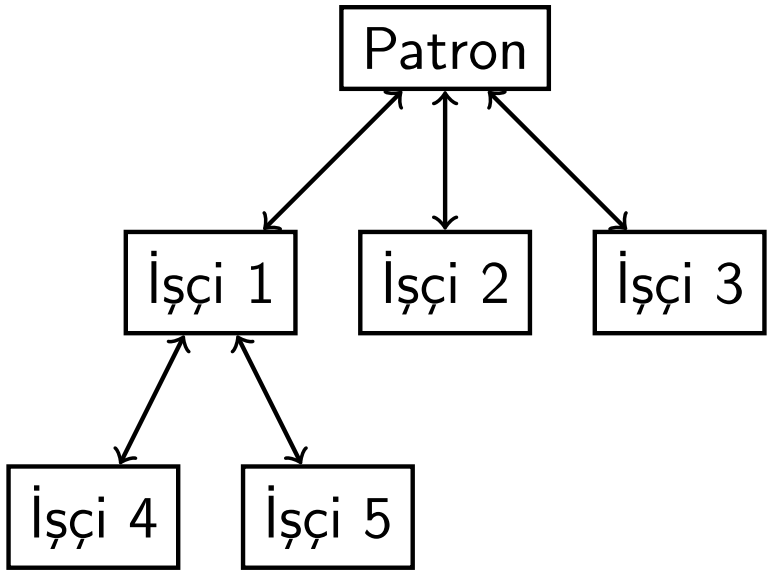
C'de Programları Modülerleştirme I

- C'de programları modülerleştirmek için C standart kütüphanesindeki fonksiyonlar ile kullanıcı tanımlı fonksiyonlar birleştirilir
- C standart kütüphanesi matematiksel işlemler, string manipülasyonu, karakter manipülasyonu ve girdi/çıktı gibi bir çok işlem için fonksiyonlar barındırır
- Ön tanımlı fonksiyonlar bir çok anlamda işimizi kolaylaştırır
- Mümkün olduğunda, yenilerini yazmak yerine mevcut fonksiyonlar kullanılmalıdır
  - Tekerleği yeniden icat etmekten kaçınılmalı
- C standart kütüphane fonksiyonları, uzmanlar tarafından yazılmış ve iyi test edilmiş fonksiyonlardır
- Belirli işleri yapmak için fonksiyonlar tanımlayabilirsiniz
- Fonksiyonlar, **fonksiyon çağrısı** ile **çalıştırılır**

C'de Programları Modülerleştirme II

- Fonksiyon çağrısı, fonksiyonun adı ve fonksiyona giden bilgileri(argüman) içerir
- Fonksiyonların çalışma şekli bir patronun(**çağıran fonksiyon** ya da **çağıran**) bir işçiye(**çağrılan fonksiyon**) bir iş yapmasını ve bitince kendisine rapor vermesini ister.
  - Örneğin ekranda veri göstermek isteyen bir fonksiyon `printf` işçi fonksiyonunu çağırır
  - `printf`, veriyi gösterir ve rapor verir(**geri döner**, **return**)
- Aşağıdaki diyagram patron fonksiyonun hiyerarşik olarak birden fazla işçi fonksiyon ile haberleşmesini gösterir

C'de Programları Modülerleştirme III



Math Kütüphane Fonksiyonları I

- `math.h` başlığında tanımlıdır
- Örneğin `900.0`'ün karekökünü yazdırmak için `printf("%.2f", sqrt(900.0));` ifadesi kullanılılr
- Üstteki komut çalışınca, önce `sqrt` fonksiyonu çağrılır, `sqrt` karekökü `30.0` olarak hesaplar ve döndürür, `printf` dönen değeri ekrana yazdırır
- Fonksiyon argümanları sabitler, değişkenler veya ifadeler olabilir
- Örneğin: `c=13.0`, `d=3.0` ve `4.0` olursa
  - `printf("%.2f", sqrt(c + d * f));` komutu `5.00` yazdırır
- Aşağıdaki tabloda C matematik fonksiyonlarının özeti bulunmaktadır

Math Kütüphane Fonksiyonları II

Fonksiyon	Açıklama	Örnek
`sqrt(x)`	x'in karekökü	`sqrt(900.0)` → `30.0` `sqrt(9.0)` → `3.0`
`cbrt(x)`	x'in küp kökü	`cbrt(27.0)` → `3.0` `cbrt(-8.0)` → `-2.0`
`exp(x)`	exponansiyel fonksiyon e^x	`exp(1.0)` → `2.718282` `exp(2.0)` → `7.389056`
`log(x)`	x'in doğal logaritması(e taban)	`log(2.718282)` → `1.0` `log(7.389056)` → `2.0`

`log10(x)`	x'in logaritması(10 taban)	`log10(1.0)` → `0.0`
		`log(10.0)` → `1.0`
		`log10(100.0)` → `2.0`

`fabs(x)`	x'in mutlak değeri(kayan noktalı ondalıklı)	`fabs(13.5)` → `13.5`
		`fabs(0.0)` → `0.0`
		`fabs(-13.5)` → `13.5`

Math Kütüphane Fonksiyonları III

`ceil(x)`	x'i yukarı yuvarlar(tamsayıya)	`ceil(9.2)` → `10.0` `ceil(-9.8)` → `-9.0`
`floor(x)`	x'i aşağı yuvarlar(tamsayıya)	`floor(9.2)` → `9.0` `floor(-9.8)` → `-10.0`
`pow(x, y)`	x üzeri y x^y	`pow(2, 7)` → `128.0` `pow(9, .5)` → `3.0`
`fmod(x, y)`	$\frac{x}{y}$ bölümünden kalanı ondalıklı olarak verir	`fmod(13.657, 2.333)` → `1.992`
`sin(x)`	x'in trigonometrik sinüsü(radyan cinsinden)	`sin(0.0)` → `0.0`
`cos(x)`	x'in trigonometrik kosinüsü(radyan cinsinden)	`cos(0.0)` → `1.0`
`tan(x)`	x'in trigonometrik tanjantı(radyan cinsinden)	`tan(0.0)` → `0.0`

Fonksiyonlar I

- **Fonksiyonlar**
  - Fonksiyonlar bir programı modüler hale getirmenizi sağlar. Birçok fonksiyon içeren programlarda, `main` genellikle programın ana iş yükünü yapan fonksiyon çağrılarından oluşan bir grup olarak uygulanır.
- **Programların Fonksiyonlara Ayrılması**
  - Bir programı fonksiyonlara ayırmanın birkaç motivasyonu vardır. Böl ve fethet yaklaşımı, program geliştirmeyi daha yönetilebilir hale getirir.
  - Bir diğer motivasyon, mevcut fonksiyonları kullanarak yeni programlar oluşturmaktır. Bu tür yazılım yeniden kullanımı, C'den türetilen nesne yönelimli programlama dillerinde (C++, Java, C#, Objective-C ve Swift) önemli bir kavramdır.
  - İyi fonksiyon isimlendirme ve tanımlama ile belirli görevleri gerçekleştiren standartlaştırılmış fonksiyonlardan oluşan programlar oluşturabilirsiniz; bu, soyutlama olarak bilinir. Her printf, scanf veya pow gibi standart kütüphane fonksiyonlarını kullandığımızda soyutlamadan yararlanıyoruz.

Fonksiyonlar II

- Üçüncü bir motivasyon ise programda kod tekrardan kaçınmaktır. Kodu bir fonksiyon olarak paketlemek, diğer program noktalarından o fonksiyonu çağırarak çalıştırılmasını sağlar.

- **İyi Bir Fonksiyon Tasarlamak**
  - Her fonksiyon, tek bir iyi tanımlanmış görevi yerine getirmekle sınırlı olmalı ve fonksiyon adı bu görevi ifade etmelidir. Bu, soyutlamayı kolaylaştırır ve yazılımın yeniden kullanılabilirliğini teşvik eder.
  - Fonksiyonun ne yaptığını tanımlayacak kısa bir ad seçemiyorsanız, muhtemelen çok çeşitli görevleri yerine getiriyordur. Böyle bir fonksiyonu daha küçük fonksiyonlara ayırmak genellikle en iyi yaklaşımdır; bu işleme ayrıştırma (decomposition) denir.

Fonksiyon Tanımları

kare Fonksiyonu

```c
// kod5_1.c
// Bir fonksiyon oluşturma ve kullanma
#include <stdio.h>

int kare(int sayi); // fonksiyon prototipi

int main(void) {
    // 10 defa dön ve x'in karesini her defasında hesapla
    for (int x = 1; x <= 10; ++x)
    {
        printf("%d ", kare(x)); // fonksiyon çağrısı
    }
    puts("");
}

// kare fonksiyon tanımı parametresinin karesini döndürür
int kare(int sayi) { // sayi, fonksiyonun argümanının bir kopyasıdır
    return sayi * sayi; // sayi'nın karesini bir tamsayı olarak döndürür
}
```

```
1 4 9 16 25 36 49 64 81 100
```

Fonksiyon Tanımları I

- kare fonksiyonunun çağrılması
  - 10. satırdaki `printf("%d ", kare(x)); // fonksiyon çağrısı` ifadesi ile kare fonksiyonu çağrılır
  - `kare` fonksiyonu `x`'in değerinin bir kopyasını alır
  - `kare` fonksiyonu `sayi * sayi` değerini hesaplar ve 10. satıra gönderir(return)
  - 10. satırda `kare` fonksiyonunun sonucu `printf` fonksiyonuna gönderilir ve yazdırılır
- kare fonksiyonunun tanımı
  - 17-19 arasındaki satırlar `kare` fonksiyonunun bit adet tamsayı(int) parametre beklediğini gösterir
  - Fonksiyonun isminden önce gelen `int`, `kare` fonksiyonunun bir tamsayı(int) döndüreceğini gösterir
  - Değer geri döndürme `return` ifadesi ile yapılır
  - Fonksiyon isimleri anlamlı seçilmelidir

Fonksiyon Tanımları II

- Çok fazla parametresi olan fonksiyonlar çok fazla göreve sahip olabilirler. Bu tip fonksiyonları ayrı görevler yapan daha küçük fonksiyonlara parçalamak faydalı olacaktır

- Yerel değişkenler
  - Fonksiyon tanımında tanımlanan tüm değişkenler **yerel değişkendir**
  - Sadece fonksiyon içinden erişilebilirler
  - Fonksiyonların parametreleri de aynı zamanda yerel değişkendir
- Fonksiyon prototipi
  - `int kare(int sayi);` ifadesi bir fonksiyon prototipidir
  - 10. satırdaki fonksiyon çağrısının doğru olup olmadığını anlamak için derleyici prototipe bakar ve aşağıdakileri karşılaştırır
    - argüman sayısının doğruluğu
    - argümanların türünün doğruluğu
    - argüman türleri doğru sırada mı
    - geri dönüş türünün fonksiyonun çağrıldığı yer ile uyumlu olması

Fonksiyon Tanımları III

```
geri-dönüş-türü fonksiyon-adı(parametre-listesi) {
        komutlar
}
```

- Fonksiyon tanımının biçimi
  - Fonksiyon adı herhangi geçerli bir tanımlayıcıdır(değişken adlandırmasındaki gibi)
  - **geri-dönüş-türü** çağırana dönecek değerin türünü belirtir
  - **parametre-listesi** virgüllerle ayrılan, parametrelerin türleri ve adlarının belirtildiği listedir
  - Her parametrenin türü ayrı ayrı belirtilmelidir, aksi takdirde derleme hatası ortaya çıkar
  - Fonksiyonun parantezinin bittiği yere ; koymak hatalıdır
- Fonksiyon gövdesi
  - Süslü parantezler içinde yer alan kısma **fonksiyonun gövdesi** ismi verilir ve bir bloktur

Fonksiyon Tanımları IV

- Bloklarda yerel değişkenler tanımlanabilir ve bloklar iç içe oluşturulabilir
- Fonksiyonlar iç içe **oluşturulamaz**

- Kontrolü fonksiyondan döndürmek
  - Kontrolü devretmek için iki yol vardır
    - Eğer fonksiyon bir değer döndürmüyorsa `return;` biçiminde
    - Eğer değer döndürüyorsa `return ifade;` biçiminde

- `main` fonksiyonunun geri dönüş türü
  - `main` fonksiyonu programın doğru çalıştığını göstermek için sıfır(0) döndürür
  - Sıfır dışındaki değerler hataları temsil eder ve işletim sistemi tarafında her değerin bir anlamı vardır

maksimum Fonksiyonu

```c
// kod5_2.c
// Üç tamsayının en büyüğünü bulmak
#include <stdio.h>

int maksimum(int x, int y, int z); // fonksiyon prototipi

int main(void) {
    int sayi1 = 0; // kullanıcı tarafından girilecek ilk tamsayı
    int sayi2 = 0; // kullanıcı tarafından girilecek ikinci tamsayı
    int sayi3 = 0; // kullanıcı tarafından girilecek üçüncü tamsayı

    printf("%s", "Üç tamsayı girin: ");
    scanf("%d%d%d", &sayi1, &sayi2, &sayi3);

    // sayi1, sayi2 ve sayi3 maksimum fonksiyon çağrısının argümanlarıdır
    printf("Maksimum: %d\n", maksimum(sayi1, sayi2, sayi3));
}

```

maksimum Fonksiyonu

```
19  // maksimum fonksiyon tanımı
20  int maksimum(int x, int y, int z) {
21      int maks = x; // x'in en büyük olduğunu varsayalım
22
23      if (y > maks) { // eğer y, maks'dan büyükse
24          maks = y; // y'yi maks'a ata
25      }
26
27      if (z > maks) { // eğer z, maks'dan büyükse
28          maks = z; // z'yi maks'a ata
29      }
30
31      return maks; // maks, en büyük değer
32  }
```

maksimum Fonksiyonu

```
Üç tamsayı girin: 22 85 17
Maksimum: 85
```

```
Üç tamsayı girin: 47 32 14
Maksimum: 47
```

```
Üç tamsayı girin: 35 8 79
Maksimum: 79
```

Fonksiyon Prototipleri: Derin Bir Bakış I

- Fonksiyon prototipleri sayesinde derleyici, fonksiyonun doğru çağrılıp çağrımadığını kontrol eder
- `int maksimum(int x, int y, int z);` prototipi
  - Geri dönüş türünün tamsayı olduğunu,
  - Üç adet tamsayı parametre aldığını belirtir
- Prototipteki değişken isimleri belgelendirme amaçlıdır
  - `int maksimum(int, int, int);` biçiminde de olabilir
- Fonksiyon çağrısı, prototip ile uyuşmazsa **derleyici hatası** oluşur
  - Örneğin prototip `void maksimum(int x, int y, int z);` biçiminde olsaydı, derleyici hatası oluşurdu çünkü prototipin geri dönüş türü `void` türünde

Fonksiyon Prototipleri: Derin Bir Bakış II

- Argüman zorlaması
  - Türler örtük olarak uygun türe dönüştürülür
  - `printf("%.3f\n", sqrt(4));` örneğinde `sqrt`'nin argümanının türü tamsayıdır, fakat `sqrt` fonksiyonu `double` türünden bir parametre alır
  - C, `sqrt` fonksiyonuna değeri göndermeden önce 4.0 olarak `double` türüne örtük olarak kopyalar
  - `sqrt` örneğinde `int` türü, daha büyük değerleri saklayabilen `double` türüne aktarıldığı için sorun olmadı
  - Daha küçük türlere yapılan dönüşümlerde veri kaybı olabilir
- Karışık türdeki ifadelerde(örn: 1 int, 1 double) derleyici tüm değişkenlerin bir kopyasını "en yüksek türde" oluşturur(terfi)

Fonksiyon Prototipleri: Derin Bir Bakış III

- Örneğin en az bir adet ondalıklı sayı içeren ifadelerde
  - Eğer bir ifade `long double` ise diğer değerler de `long double` türüne dönüştürülür
  - Eğer bir ifade `double` ise diğer değerler de `double` türüne dönüştürülür
  - Eğer bir ifade `float` ise diğer değerler de `float` türüne dönüştürülür
- Eğer karışık türdeki ifade sadece tamsayılar barındırıyorsa, tamsayı terfi aritmetiği uygulanır
- Bir tür, daha düşük bir türe dönüştürülecekse, bu açık olarak yapılır ve bu işleme **cast** adı verilir
  - `double pi=3.14; int a; a=(int)pi;`
  - Bu işlem açık olarak yapılmazsa veri kaybı olur ve derleyici uyarı verir
- Eğer fonksiyonun prototipi yoksa, derleyici fonksiyonun tanımını gördüğü ilk yerde bir tane otomatik olarak oluşturur
  - Bu noktadan önce fonksiyonu çağırmak derleyici hatasına neden olur

Fonksiyon Prototipleri: Derin Bir Bakış I

Veri Türü	`printf` **Dönüşüm Spesifikasyonu**	`scanf` **Dönüşüm Spesifikasyonu**
Kayan Nokta Türleri		
`long double`	%Lf	%Lf
`double`	%f	%lf
`float`	%f	%f
Tamsayı Türleri		
`unsigned long long int`	%llu	%llu
`long long int`	%lld	%lld
`unsigned long int`	%lu	%lu
`long int`	%ld	%ld
`unsigned int`	%u	%u

Fonksiyon Prototipleri: Derin Bir Bakış II

`int`	%d	%d
`unsigned short`	%hu	%hu
`short`	%hd	%hd
`char`	%c	%c

Fonksiyon Çağrı Yığıtı ve Yığıt Çerçeveleri I

- Fonksiyonların nasıl çalıştığını anlamak için öncelikle yığıt(stack) veri yapısını anlamalıyız
  - Stack son girenin ilk çıktığı(last-in-first-out, LIFO) mantığıyla çalışır
  - Bir mutfak rafına tabakları dizmeye benzer, yeni bir tabağı en üste yerleştiririz, tabak alacağımızda da en üstten alırız
  - Ekleme için **push** ve silmek için **pop** gibi iki temel işlemi vardır
- **Fonksiyon-çağrı-yığıtı** adı verilen mekanizma arka planda fonksiyonların çalışmasını sağlar
- Bir fonksiyon çağrısı yapıldığında, o fonksiyon başka fonksiyonları da çağırabilir
- Her fonksiyon çağrısı için, kontrolü devrettiğinde çağrıran fonksiyondaki dönüş adresini bilmemiz gerekir
- Her fonksiyon çağrısında fonksiyon-çağrı-yığıtına(stack) bir kayıt eklenir(**push**), buna **yığıt çerçevesi**(stack frame) adı verilir

Fonksiyon Çağrı Yığıtı ve Yığıt Çerçeveleri II

- Yığıt çerçevesi, kontrolü devrinde gidilecek **dönüş adresini** barındırır
- Her fonksiyon ihtiyaç halinde dönüş adresini yığıtın en tepesinde bulur
- Bir fonksiyon çağrısı sonlandığında fonksiyonun yığıt çerçevesi yığıttan silinir(**pop**)
- Bilgisayardaki bellek miktarı sınırlıdır ve fonksiyon çağrı yığıtı için belli miktarda yer ayrılır. Bu yerin dolması durumunda yığıt taşması(stack overflow) gerçekleşir

Fonksiyon Çağrı Yığıtı Örneği

```c
// kod5_3.c
// kare fonksiyonunu kullanarak fonksiyon çağrı yığıtını
// ve yığıt çerçevelerini gösterme
#include <stdio.h>

int kare(int x); // kare fonksiyonu için prototip

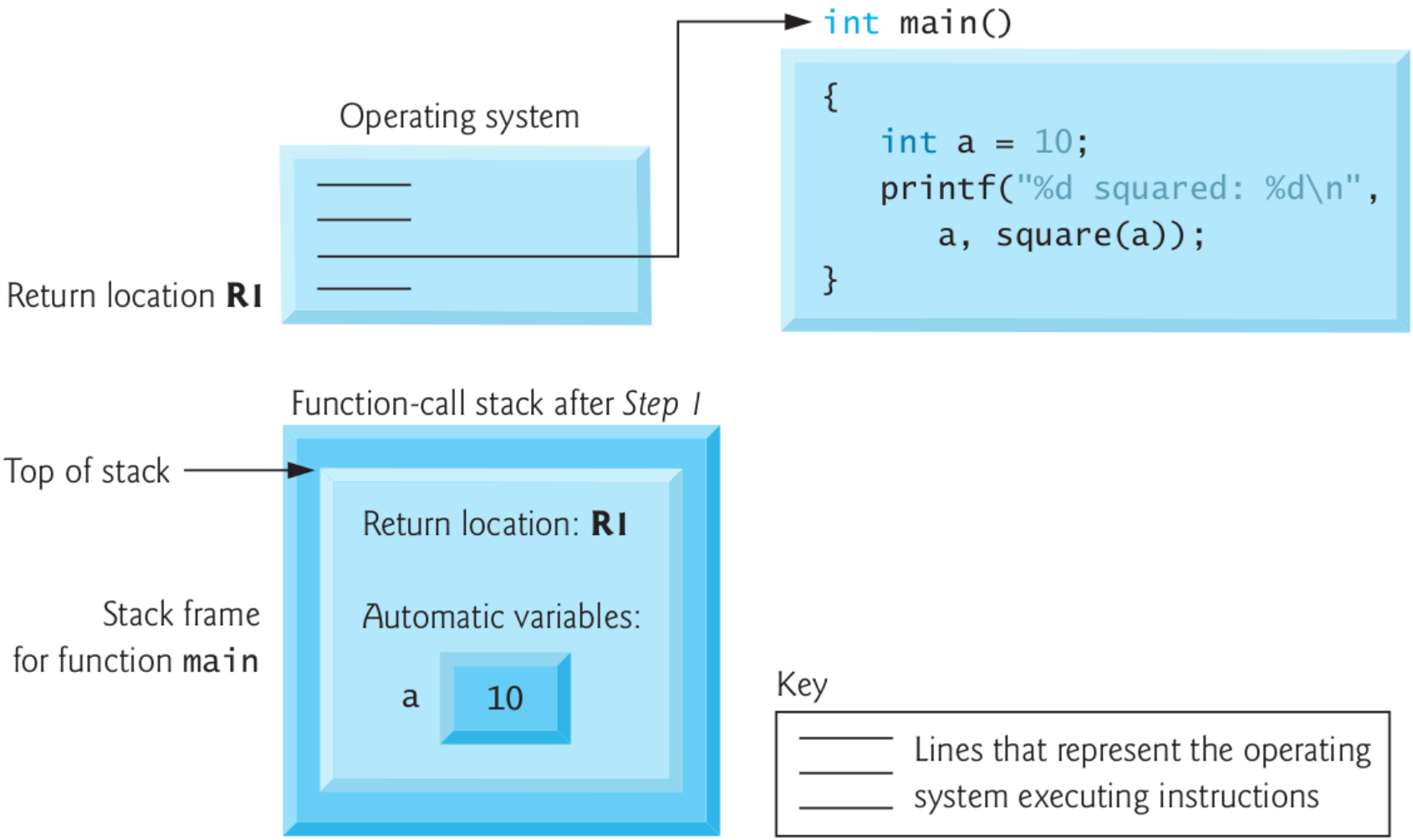
int main() {
    int a = 10; // karesi alınacak değer(main içinde yerel değişken)

    printf("%d karesi %d\n", a, kare(a)); // a'nın karesini göster
}

// bir tamsayının karesini geri döndürür
int kare(int x) { // x bir yerel değişkendir
    return x * x; // kareyi hesapla ve sonucu geri döndür
}
```

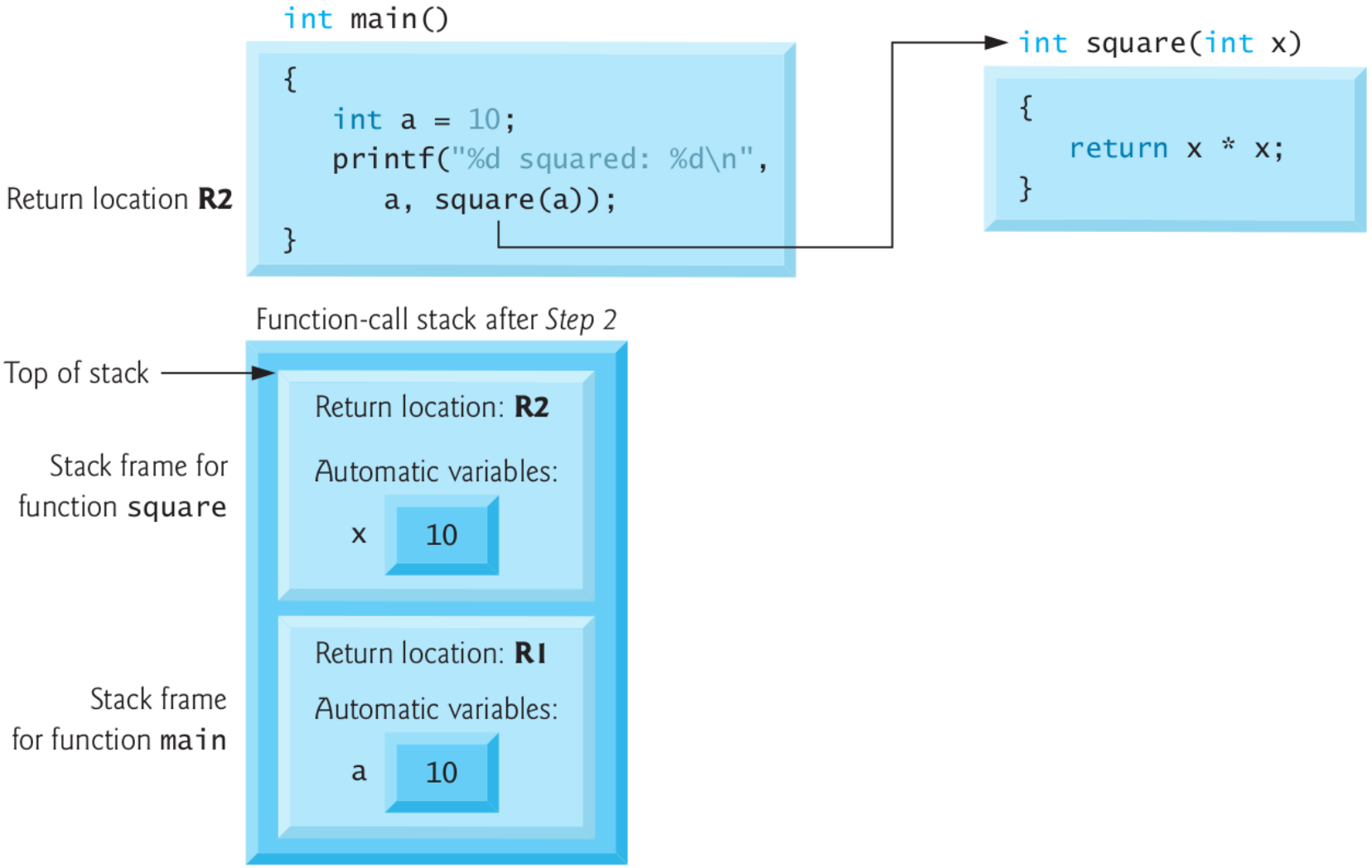
Adım 1

İşletim sistemi `main` fonksiyonu ile programı başlatır



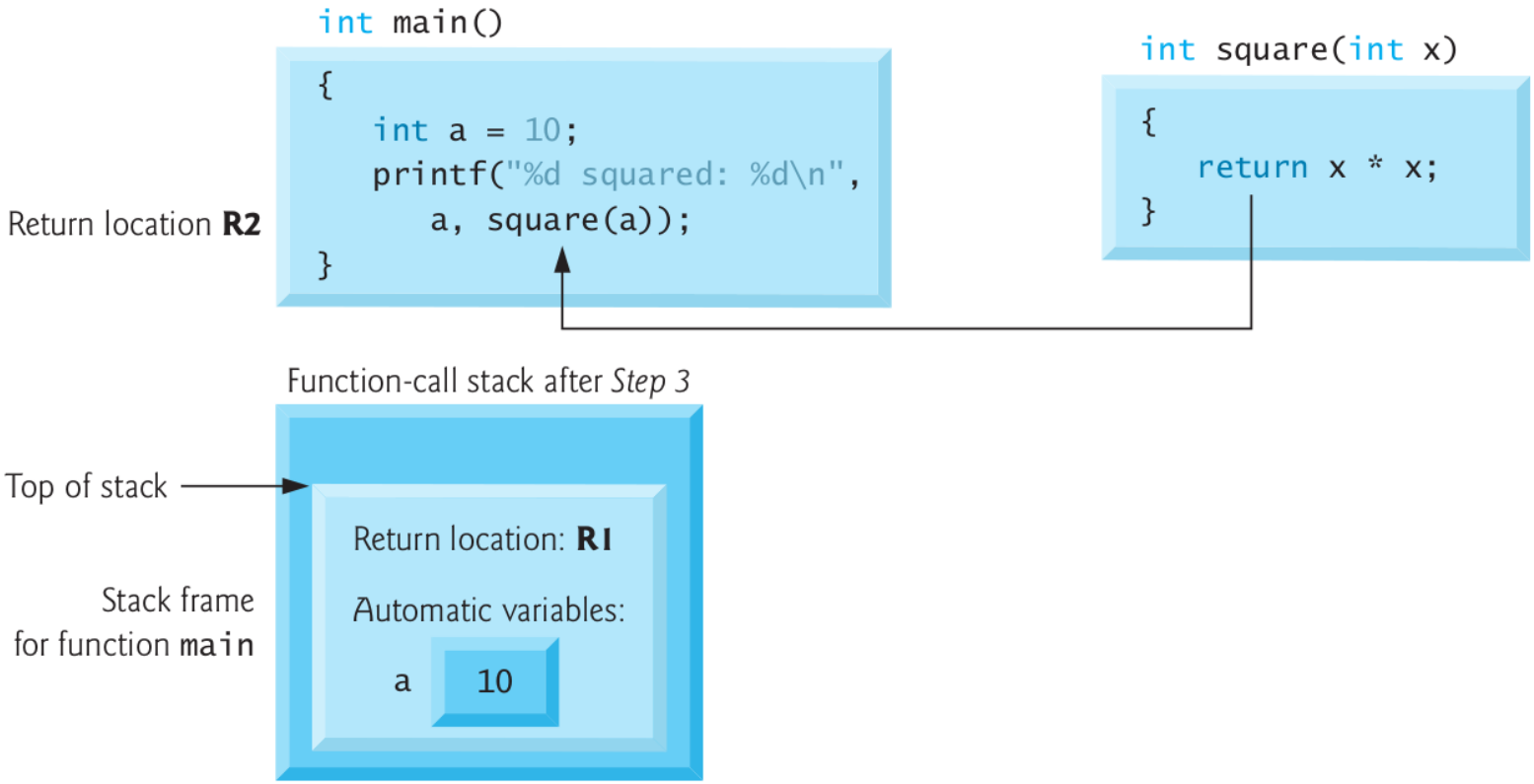
Adım 2

`main` fonksiyonu, hesaplama için `kare` fonksiyonunu çağırır



Adım 3

`kare` fonksiyonu, sonuçları `kare` fonksiyonuna döndürür



Başlıklar I

Başlık	Açıklama
Bu kitapta kullandığımız veya incelediğimiz başlıklar:	
`<assert.h>`	Program hata ayıklamasını destekleyen tanılayıcıları eklemek için bilgiler içerir.
`<ctype.h>`	Belirli özellikler için karakterleri test eden ve küçük harfleri büyük harflere veya tam tersine dönüştürebilecek fonksiyon prototiplerini içerir.
`<float.h>`	Sistem için kayan nokta boyutu limitlerini içerir.
`<limits.h>`	Sistem için tamsayı boyut limitlerini içerir.

Başlıklar II

`<math.h>`	Matematik kütüphanesi fonksiyonları için fonksiyon prototiplerini içerir.
`<signal.h>`	Program yürütülmesi sırasında oluşabilecek çeşitli koşulları ele almak için fonksiyon prototipleri ve makrolar içerir.
`<stdarg.h>`	Sayısı ve türleri bilinmeyen bir fonksiyon için argüman listesini ele almak için makrolar tanımlar.
`<stdio.h>`	Standart giriş/çıkış kütüphanesi işlevleri ve bunların kullandığı bilgilerin fonksiyon prototiplerini içerir.
`<stdlib.h>`	Sayıların metne ve metinlerin sayıya dönüştürülmesi, bellek tahsisi, rastgele sayılar ve diğer yardımcı fonksiyonlar için fonksiyon prototiplerini içerir.

Başlıklar III

`<string.h>`	Dize(String) işleme fonksiyonları için fonksiyon prototiplerini içerir.
`<time.h>`	Tarih ve zamanı işlemek için fonksiyon prototipleri ve türlerini içerir.
Diğer başlıklar:	
`<errno.h>`	Hata koşullarını bildirmek için faydalı olan makroları tanımlar.
`<locale.h>`	Programın çalıştığı geçerli yerel ayara göre değiştirilebilen fonksiyon prototiplerini ve diğer bilgileri içerir. Yerel ayar kavramı, tarihler, saatler, para birimleri ve büyük sayılar gibi farklı veriler için sistemin farklı düzenlemeler ele almasını sağlar.

Başlıklar IV

`<setjmp.h>`	Normal fonksiyon çağrısı ve dönüş dizisini atlayarak fonksiyonların çağrılmasına olanak tanıyan fonksiyonlar için fonksiyon prototiplerini içerir.
`<stddef.h>`	C tarafından kullanılan ortak tür tanımlarını içerir.

Başlıklar V

Argümanları Değer ve Referans Olarak Geçme

- Programlama dillerinde argümanları değer olarak geçme(pass by value) ve referans olarak geçme(pass by reference) gibi yollar vardır
- Değer olarak geçmede, argümanın bir kopyası oluşturulur ve fonksiyona gönderilir
  - Fonksiyon içinde değer değiştirilse bile çağıran fonksiyonda bir etkisi olmayacaktır
- C'de argümanlar değer olarak geçmeyi kullanırlar
- İşaretçiler ünitesinde referans olarak geçme anlatılacaktır

Rastgele Sayı Üretimi

```c
// kod5_4.c
// 1 + rand() % 6 ile üretilen ötelenmiş, ölçeklenmiş rastgele sayılar
#include <stdio.h>
#include <stdlib.h>

int main(void) {

    for (int i = 0; i < 10; ++i) {
        printf("%d ", 1 + (rand() % 6)); // rastgele zar değerini göster
    }

    puts("");
}
```

Zarı 60M Defa Atmak

```
1  // kod5_5.c
2  // Altı yüzlü bir zarı 60.000.000 defa atmak
3  #include <stdio.h>
4  #include <stdlib.h>
5
6  int main(void) {
7      int frekans1 = 0; // 1 atma sayacı
8      int frekans2 = 0; // 2 atma sayacı
9      int frekans3 = 0; // 3 atma sayacı
10     int frekans4 = 0; // 4 atma sayacı
11     int frekans5 = 0; // 5 atma sayacı
12     int frekans6 = 0; // 6 atma sayacı
13
14     // 60000000 defa zar at ve sonuçları özetle
15     for (int atis = 1; atis <= 60000000; ++atis) {
16         int yuz = 1 + rand() % 6; // 1'den 6'ya kadar rastgele sayı
17
```

Zarı 60M Defa Atmak

```
18          // yüzü belirle ve uygun sayacı artır
19          switch (yuz) {
20          case 1: // 1 atıldı
21              ++frekans1;
22              break;
23          case 2: // 2 atıldı
24              ++frekans2;
25              break;
26          case 3: // 3 atıldı
27              ++frekans3;
28              break;
29          case 4: // 4 atıldı
30              ++frekans4;
31              break;
32          case 5: // 5 atıldı
33              ++frekans5;
34              break;
35          case 6: // 6 atıldı
36              ++frekans6;
37              break; // opsiyonel
38          }
39      }
```

Zarı 60M Defa Atmak

```
41      // sonuçları tablo olarak göster
42      printf("%s%13s\n", "Yüz", "Frekans");
43      printf("  1%13d\n", frekans1);
44      printf("  2%13d\n", frekans2);
45      printf("  3%13d\n", frekans3);
46      printf("  4%13d\n", frekans4);
47      printf("  5%13d\n", frekans5);
48      printf("  6%13d\n", frekans6);
49  }
```

```
Yüz       Frekans
  1       9998142
  2       9999581
  3      10000326
  4      10003300
  5       9997641
  6      10001010
```

Zar Atmayı Rastgeleleştirmek

```c
// kod5_6.c
// Zar atmayı rastgeleleştirme
#include <stdio.h>
#include <stdlib.h>

int main(void) {
    printf("%s", "Tohum değerini girin: ");
    int tohum = 0; // rastgele sayı üretecine tohum olarak verilecek değer
    scanf("%d", &tohum);

    srand(tohum); // rastgele sayı üretecini tohumla

    for (int i = 0; i < 10; ++i) {
        printf("%d ", 1 + (rand() % 6)); // rastgele zar değerini göster
    }

    puts("");
}
```

Rastgele Sayı Simülasyonu Vaka Çalışması: Bir Casino Oyunu Oluşturma

Barbut Oyunu

Bir oyuncu iki zar atar. Her zarın altı yüzü vardır. Bu yüzlerde 1, 2, 3, 4, 5 ve 6 nokta bulunur. Zarlar durduktan sonra, iki yukarı yüzdeki noktaların toplamı hesaplanır. İlk atışta toplam 7 veya 11 gelirse, oyuncu kazanır. İlk atışta toplam 2, 3 veya 12 gelirse (buna "barbut" denir), oyuncu kaybeder (yani, "kumarhane" kazanır). İlk atışta toplam 4, 5, 6, 8, 9 veya 10 olursa, bu toplam oyuncunun "puanı" olur. Kazanmak için, "puanınızı yapana" kadar zar atmaya devam etmelisiniz. Oyuncu, puanı yapmadan önce 7 atarsa kaybeder.

Barbut Oyunu

```c
// kod5_7.c
// Barbut oyununun simülasyonu
#include <stdio.h>
#include <stdlib.h>
#include <time.h> // time fonksiyonu prototipini içerir

enum Durum {DEVAM, KAZANDI, KAYBETTI}; // sabitler oyun durumunu temsil eder

int zarAt(void); // zarAt fonksiyon prototipi

int main(void) {
    srand(time(NULL)); // mevcut zamana göre rastgeleleştir

    int puanim = 0; // kazanmak için oyuncu bu puanı atmalı
    enum Durum oyunDurumu = DEVAM; // DEVAM, KAZANDI, KAYBETTI olabilir
    int toplam = zarAt(); // zarın ilk defa atılması

```

Barbut Oyunu

```
18      // zarların toplamına göre oyun durumunu belirle
19      switch (toplam)
20      {
21      // ilk atışta kazanma
22      case 7: // 7 kazanır
23      case 11: // 11 kazanır
24          oyunDurumu = KAZANDI;
25          break;
26      // ilk atışta kaybetma
27      case 2: // 2 kaybeder
28      case 3: // 3 kaybeder
29      case 12: // 12 kaybeder
30          oyunDurumu = KAYBETTI;
31          break;
32      // puanı hatırla
33      default:
34          oyunDurumu = DEVAM; // oyuncu atmaya devam etmeli
35          puanim = toplam; // puanı hatırla
36          printf("Puan: %d\n", puanim);
37          break; // opsiyonel
38      }
```

Barbut Oyunu

```
39      // oyun tamamlanmadığı sürece
40      while (DEVAM == oyunDurumu) { // oyuncu atmaya devam etmeli
41          toplam = zarAt(); // yeniden zar at
42
43          // oyun durumunu belirle
44          if (toplam == puanim) { // puanı atarak kazanır
45              oyunDurumu = KAZANDI;
46          }
47          else if (7 == toplam) { // 7 atarak kaybeder
48              oyunDurumu = KAYBETTI;
49          }
50      }
51
52      // kazandı ya da kaybetti mesajı göster
53      if (KAZANDI == oyunDurumu) { // oyuncu kazandı mı?
54          puts("Oyuncu kazandı");
55      }
56      else { // oyuncu kaybeder
57          puts("Oyuncu kaybetti");
58      }
59  }
```

Barbut Oyunu

```
60
61  // zar at, toplamı hesapla ve sonuçları göster
62  int zarAt(void) {
63      int zar1 = 1 + (rand() % 6); // rastgele zar1 değeri seç
64      int zar2 = 1 + (rand() % 6); // rastgele zar2 değeri seç
65
66      // bu atışın sonuçlarını göster
67      printf("Oyuncunun zarları: %d + %d = %d\n", zar1, zar2, zar1 + zar2);
68      return zar1 + zar2; // zarların toplamını döndür
69  }
```

Barbut Oyunu

```
Oyuncunun zarları: 6 + 1 = 7
Oyuncu kazandı
```

```
Oyuncunun zarları: 1 + 5 = 6
Puan: 6
Oyuncunun zarları: 1 + 3 = 4
Oyuncunun zarları: 5 + 3 = 8
Oyuncunun zarları: 6 + 6 = 12
Oyuncunun zarları: 3 + 2 = 5
Oyuncunun zarları: 4 + 5 = 9
Oyuncunun zarları: 1 + 3 = 4
Oyuncunun zarları: 3 + 4 = 7
Oyuncu kaybetti
```

```
Oyuncunun zarları: 3 + 6 = 9
Puan: 9
Oyuncunun zarları: 6 + 6 = 12
Oyuncunun zarları: 2 + 3 = 5
Oyuncunun zarları: 3 + 6 = 9
Oyuncu kazandı
```

```
Oyuncunun zarları: 6 + 6 = 12
Oyuncu kaybetti
```